

Virtual Machines & Docker

Implementation Report

by

Shat Chakra Pawar Amgothu

Table of Contents

1. Project Overview and Environment Setup.....	3
2. Task 2 Install and Use Linux Operating Systems	3
2.1 Creation and Execution of Virtual Machines	3
2.2 Ubuntu User Management and Python Program Execution.....	4
2.3 Debian Sudo Configuration and Synaptic Package Manager	5
3. Task 3 Install and Use Docker in Ubuntu	8
3.1 Docker Installation and Service Verification.....	8
3.2 Docker Hub Authentication and Hello World Container	9
3.3 Running an Ubuntu Container and Inspecting Containers	9
3.4 Creating a Custom Command Inside the Ubuntu Container	10
4. Task 4 Create a Python Container.....	11
4.1 Python Program Preparation.....	11
4.2 Dockerfile Creation and Image Build.....	12
4.3 Running the Python Container	13
5. Task 5 Flask Python Container with Template	13
5.1 Project Structure Setup	13
5.2 Flask Application Implementation	14
5.3 Dockerfile Creation Build and Run.....	15
5.4 Browser Verification.....	16
6. Conclusion	17

1. Project Overview and Environment Setup

The purpose of this project was to develop practical skills in Linux operating systems virtualization and container-based application deployment. The tasks focused on installing and configuring Linux virtual machines managing users and software packages and working with Docker to run and deploy containerized applications.

Oracle VirtualBox was used as the virtualization platform. Two Linux operating systems were installed and configured Ubuntu and Debian. These virtual machines were used to perform all required tasks from Task 2 to Task 5. Docker was installed on the Ubuntu virtual machine and used to execute standard Docker images as well as custom containerized Python and Flask applications.

This report documents the implementation of Tasks 2 to Task 5. Evidence for each task is provided through command line outputs and screenshots which are referenced in the relevant sections.

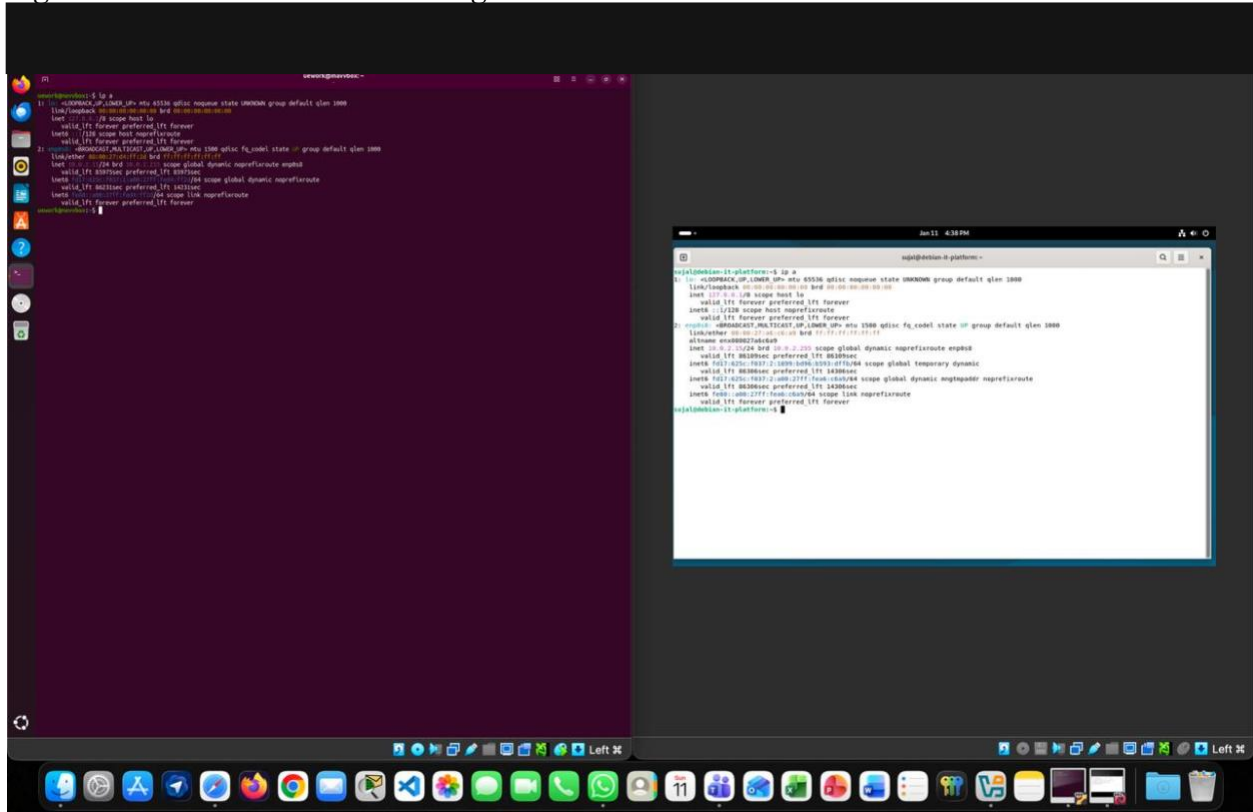
2. Task 2 Install and Use Linux Operating Systems

2.1 Creation and Execution of Virtual Machines

Two Linux virtual machines were created using Oracle VirtualBox. The first virtual machine was installed with Ubuntu, and the second virtual machine was installed with Debian version 13 Trixie. Both systems were configured with graphical desktop environments and network access.

After installation both virtual machines were started and verified to be running at the same time. This confirmed that the host system could run multiple Linux operating systems simultaneously.

Figure 1 Ubuntu and Debian running at the same time



2.2 Ubuntu User Management and Python Program Execution

All tasks in this section were performed using the command line interface only.

Two users were created on the Ubuntu system. One user was configured as an administrative user with sudo privileges, and the second user was created as a normal user. The administrative user was added to the sudo group and group membership was verified using terminal commands.

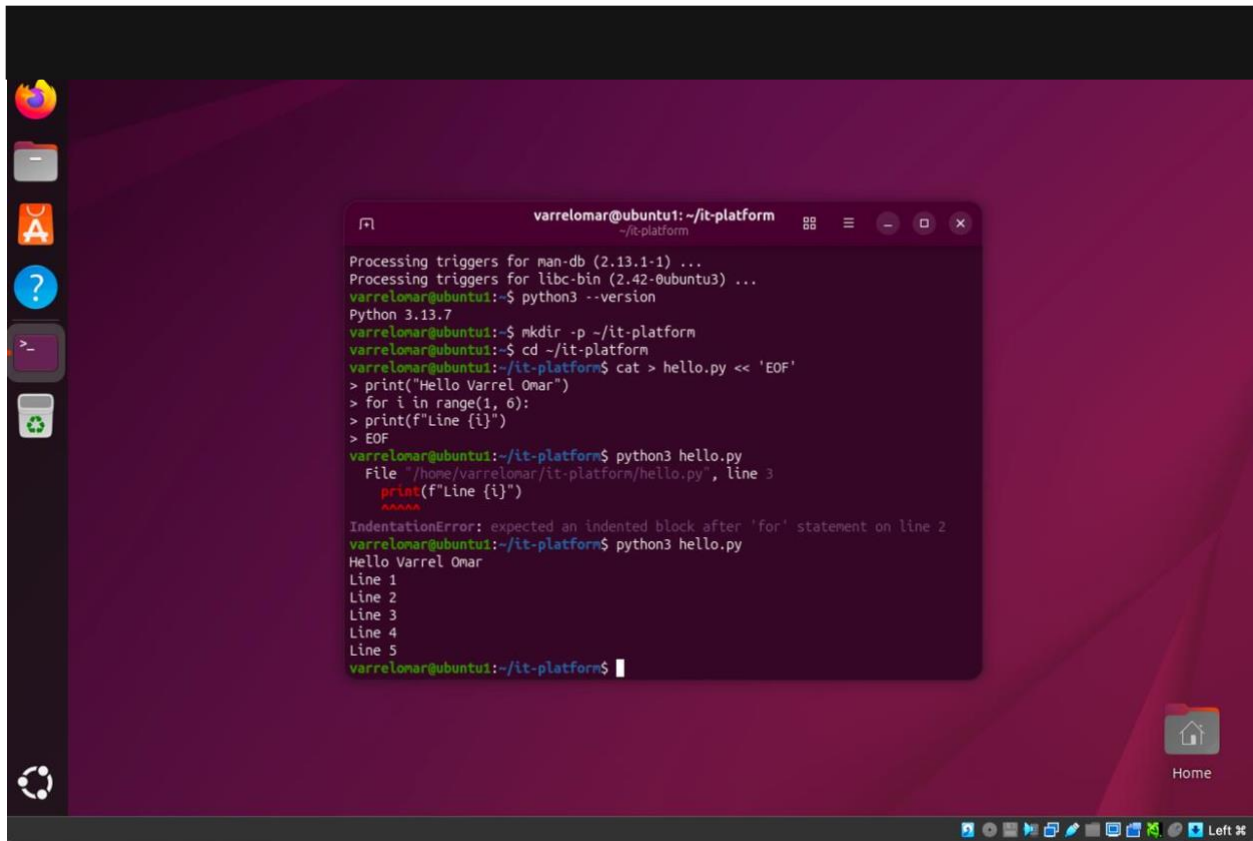


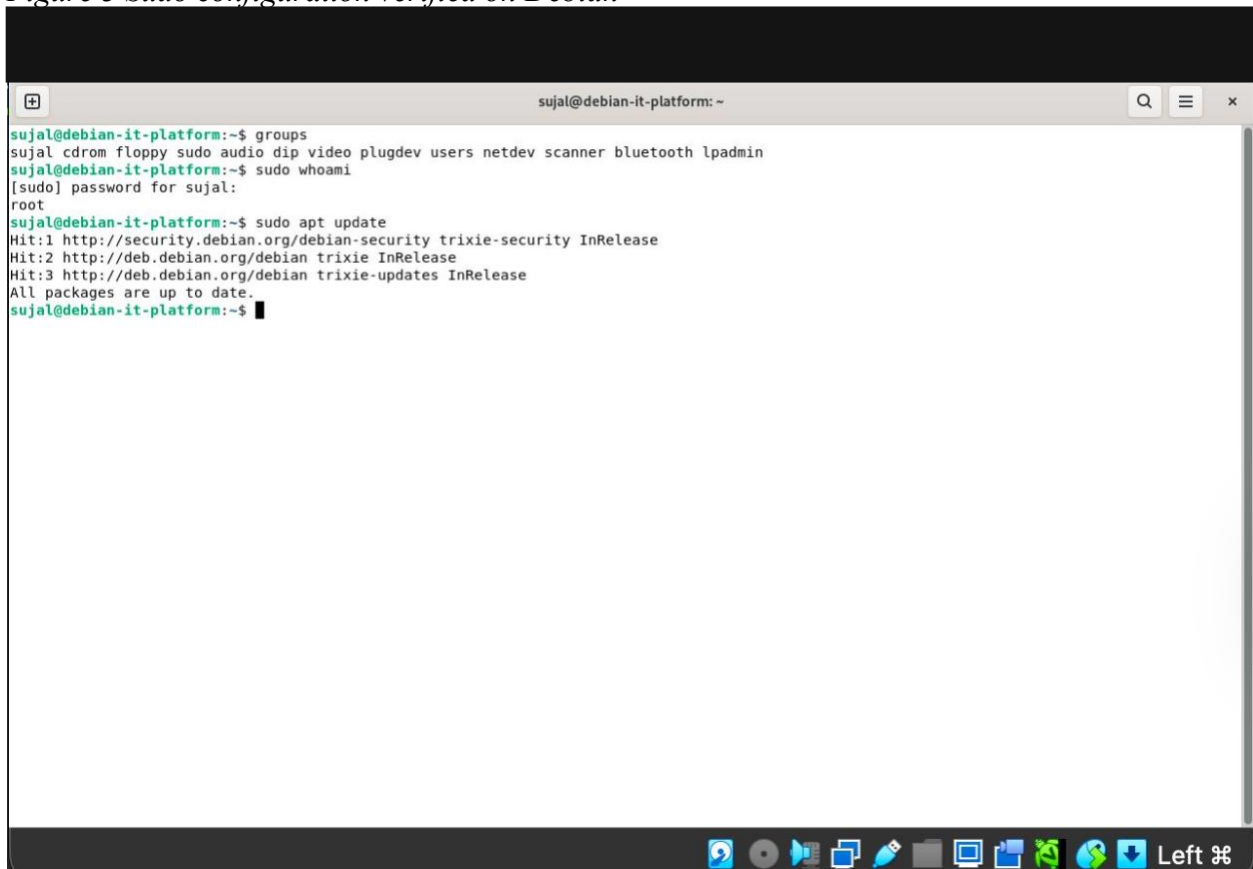
Figure 2 Python program executed on Ubuntu using command line

Python was then installed on the Ubuntu system using the package manager. The installed Python version was verified using the command line. A small Python program was created and executed directly from the terminal. The successful execution of the program confirmed that Python was correctly installed and functional.

2.3 Debian Sudo Configuration and Synaptic Package Manager

On the Debian virtual machine sudo functionality was configured. The sudo package was installed and the main user account was added to the sudo group. Administrative access was verified using a command that confirmed elevated privileges.

Figure 3 Sudo configuration verified on Debian

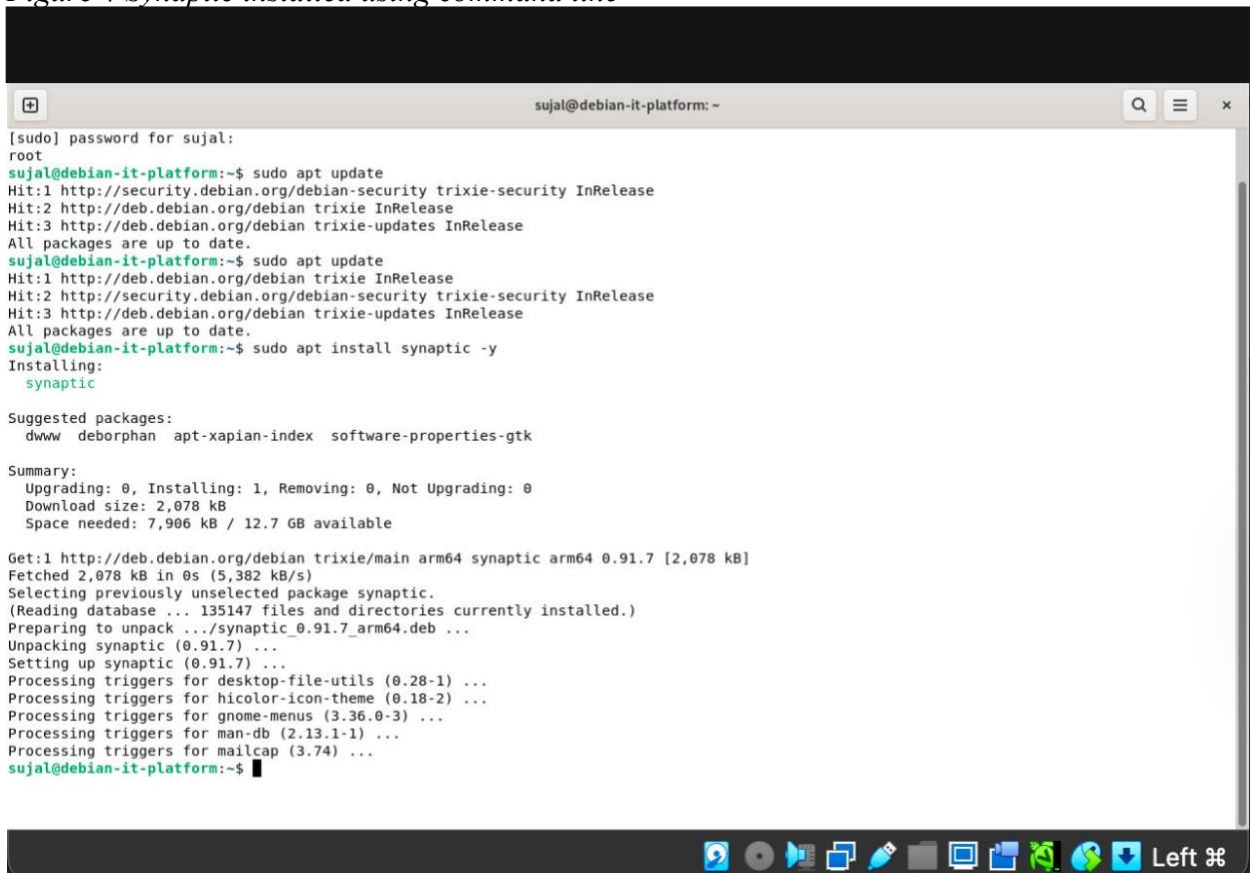
A terminal window titled 'sujal@debian-it-platform: ~' with search, menu, and close icons. The terminal shows the following commands and output:

```
sujal@debian-it-platform:~$ groups
sujal cdrom floppy sudo audio dip video plugdev users netdev scanner bluetooth lpadmin
sujal@debian-it-platform:~$ sudo whoami
[sudo] password for sujal:
root
sujal@debian-it-platform:~$ sudo apt update
Hit:1 http://security.debian.org/debian-security trixie-security InRelease
Hit:2 http://deb.debian.org/debian trixie InRelease
Hit:3 http://deb.debian.org/debian trixie-updates InRelease
All packages are up to date.
sujal@debian-it-platform:~$
```

The terminal window has a dark theme and a taskbar at the bottom with various system icons and the text 'Left ⌘'.

The Synaptic Package Manager was installed using the command line. After installation Synaptic was launched with administrative privileges. This confirmed correct installation and integration of a graphical package management tool.

Figure 4 Synaptic installed using command line



```
suja1@debian-it-platform: ~
[sudo] password for suja1:
root
suja1@debian-it-platform:~$ sudo apt update
Hit:1 http://security.debian.org/debian-security trixie-security InRelease
Hit:2 http://deb.debian.org/debian trixie InRelease
Hit:3 http://deb.debian.org/debian trixie-updates InRelease
All packages are up to date.
suja1@debian-it-platform:~$ sudo apt update
Hit:1 http://deb.debian.org/debian trixie InRelease
Hit:2 http://security.debian.org/debian-security trixie-security InRelease
Hit:3 http://deb.debian.org/debian trixie-updates InRelease
All packages are up to date.
suja1@debian-it-platform:~$ sudo apt install synaptic -y
Installing:
synaptic

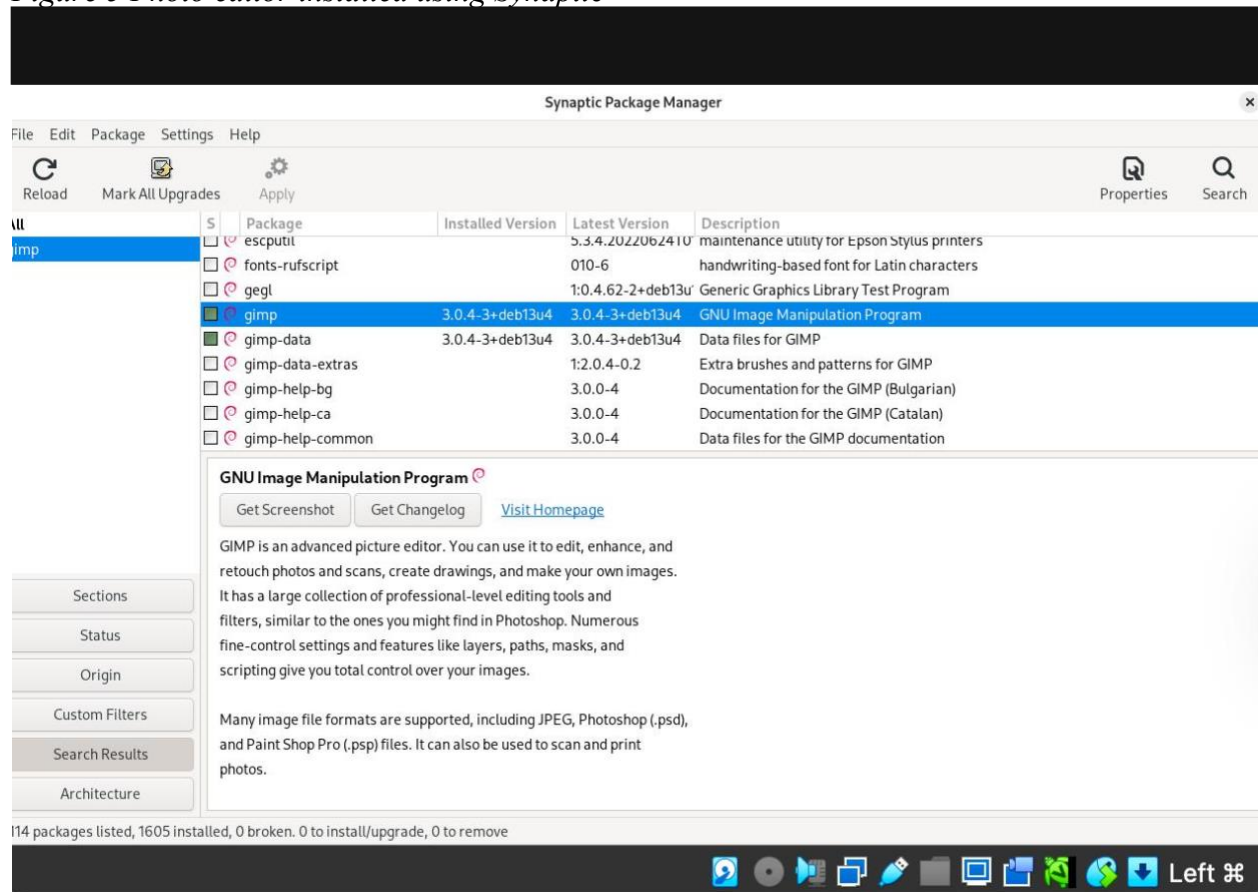
Suggested packages:
dwww deborphan apt-xapian-index software-properties-gtk

Summary:
Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 0
Download size: 2,078 kB
Space needed: 7,906 kB / 12.7 GB available

Get:1 http://deb.debian.org/debian trixie/main arm64 synaptic arm64 0.91.7 [2,078 kB]
Fetched 2,078 kB in 0s (5,382 kB/s)
Selecting previously unselected package synaptic.
(Reading database ... 135147 files and directories currently installed.)
Preparing to unpack .../synaptic_0.91.7_arm64.deb ...
Unpacking synaptic (0.91.7) ...
Setting up synaptic (0.91.7) ...
Processing triggers for desktop-file-utils (0.28-1) ...
Processing triggers for hicolor-icon-theme (0.18-2) ...
Processing triggers for gnome-menus (3.36.0-3) ...
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for mailcap (3.74) ...
suja1@debian-it-platform:~$
```

Using Synaptic a graphical photo editing application was installed. The installed application was successfully launched from the system menu which confirmed correct software installation using a graphical interface.

Figure 5 Photo editor installed using Synaptic

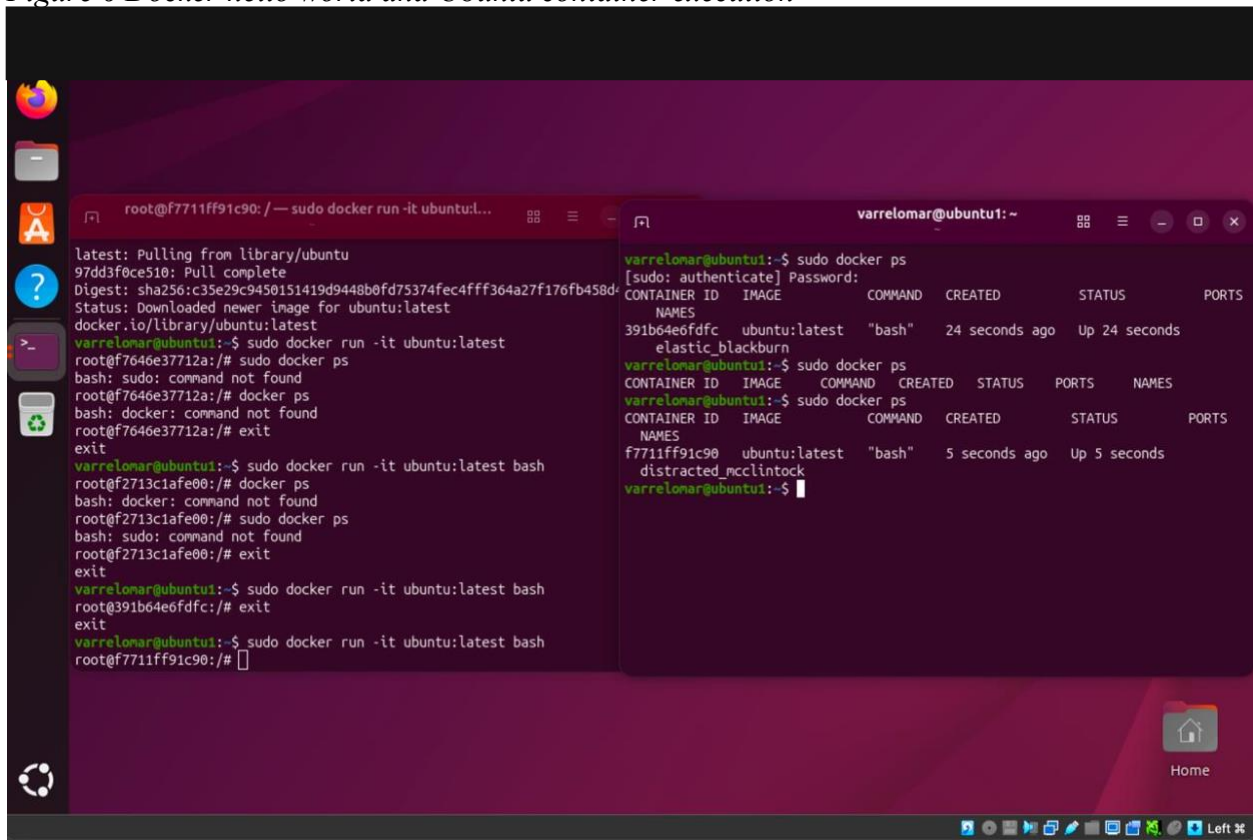


3. Task 3 Install and Use Docker in Ubuntu

3.1 Docker Installation and Service Verification

Docker was installed on the Ubuntu virtual machine using the system package manager. After installation the Docker service was enabled and started. The Docker version was verified using the command line to confirm that Docker was installed correctly. A container listing command was executed to verify that the Docker service was running.

Figure 6 Docker hello world and Ubuntu container execution



```
root@f7711ff91c90: /# sudo docker run -it ubuntu:latest
latest: Pulling from library/ubuntu
97dd3f0ce510: Pull complete
Digest: sha256:c35e29c9450151419d9448b0fd75374fec4fff364a27f176fb458d
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest
varrelomar@ubuntu1:~$ sudo docker run -it ubuntu:latest
root@f7646e37712a: /# sudo docker ps
bash: sudo: command not found
root@f7646e37712a: /# docker ps
bash: docker: command not found
root@f7646e37712a: /# exit
exit
varrelomar@ubuntu1:~$ sudo docker run -it ubuntu:latest bash
root@f2713c1afe00: /# docker ps
bash: docker: command not found
root@f2713c1afe00: /# sudo docker ps
bash: sudo: command not found
root@f2713c1afe00: /# exit
exit
varrelomar@ubuntu1:~$ sudo docker run -it ubuntu:latest bash
root@391b64e6dfc: /# exit
exit
varrelomar@ubuntu1:~$ sudo docker run -it ubuntu:latest bash
root@f7711ff91c90: /#
```

```
varrelomar@ubuntu1:~$ sudo docker ps
[sudo: authenticate] Password:
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
NAMES
391b64e6dfc    ubuntu:latest  "bash"                   24 seconds ago Up 24 seconds
elastic_blackburn
varrelomar@ubuntu1:~$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
NAMES
varrelomar@ubuntu1:~$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
NAMES
f7711ff91c90   ubuntu:latest  "bash"                   5 seconds ago  Up 5 seconds
distracted_mcclintock
varrelomar@ubuntu1:~$
```

3.2 Docker Hub Authentication and Hello World Container

Authentication with Docker Hub was performed using the docker login command. After successful authentication the official hello world Docker image was executed. The output confirmed that Docker was able to pull images from Docker Hub and run containers successfully.

3.3 Running an Ubuntu Container and Inspecting Containers

The official Ubuntu image was pulled from Docker Hub. An interactive Ubuntu container was started using a Bash shell. While the container was running the list of active containers was displayed. After exiting the container both running and stopped containers were listed to verify container lifecycle behavior.

Figure 7 Custom command created inside Docker container

```
nead save log
root@f7711ff91c90:/# cat > /usr/local/bin/tulis << 'EOF'
> #!/bin/bash
> /bin/ls -la "$@"
> EOF
root@f7711ff91c90:/# chmod +x /usr/local/bin/tulis
root@f7711ff91c90:/# tulis /
total 60
drwxr-xr-x  1 root root 4096 Jan 11 14:36 .
drwxr-xr-x  1 root root 4096 Jan 11 14:36 ..
-rwxr-xr-x  1 root root    0 Jan 11 14:36 .dockerenv
lrwxrwxrwx  1 root root    7 Apr 22  2024 bin -> usr/bin
drwxr-xr-x  2 root root 4096 Apr 22  2024 boot
drwxr-xr-x  5 root root  360 Jan 11 14:36 dev
drwxr-xr-x  1 root root 4096 Jan 11 14:36 etc
drwxr-xr-x  3 root root 4096 Oct 13 14:12 home
lrwxrwxrwx  1 root root    7 Apr 22  2024 lib -> usr/lib
drwxr-xr-x  2 root root 4096 Oct 13 14:06 media
drwxr-xr-x  2 root root 4096 Oct 13 14:06 mnt
drwxr-xr-x  2 root root 4096 Oct 13 14:06 opt
dr-xr-xr-x 255 root root    0 Jan 11 14:36 proc
drwx-----  2 root root 4096 Oct 13 14:12 root
drwxr-xr-x  4 root root 4096 Oct 13 14:12 run
lrwxrwxrwx  1 root root    8 Apr 22  2024/sbin -> usr/sbin
drwxr-xr-x  2 root root 4096 Oct 13 14:06 srv
```

3.4 Creating a Custom Command Inside the Ubuntu Container

Inside the running Ubuntu container, a custom command was created to replicate the functionality of the `ls la` command. A shell script was created in the system path and made executable. The new command was then executed successfully inside the container.

This task demonstrated the ability to modify the container environment and create custom executable commands.

Figure 8 Custom command executed inside Docker container

```
-rwxr-xr-x 1 root root 0 Jan 11 14:36 .dockerenv
lrwxrwxrwx 1 root root 7 Apr 22 2024 bin -> usr/bin
drwxr-xr-x 2 root root 4096 Apr 22 2024 boot
drwxr-xr-x 5 root root 360 Jan 11 14:36 dev
drwxr-xr-x 1 root root 4096 Jan 11 14:36 etc
drwxr-xr-x 3 root root 4096 Oct 13 14:12 home
lrwxrwxrwx 1 root root 7 Apr 22 2024 lib -> usr/lib
drwxr-xr-x 2 root root 4096 Oct 13 14:06 media
drwxr-xr-x 2 root root 4096 Oct 13 14:06 mnt
drwxr-xr-x 2 root root 4096 Oct 13 14:06 opt
dr-xr-xr-x 255 root root 0 Jan 11 14:36 proc
drwx----- 2 root root 4096 Oct 13 14:12 root
drwxr-xr-x 4 root root 4096 Oct 13 14:12 run
lrwxrwxrwx 1 root root 8 Apr 22 2024 sbin -> usr/sbin
drwxr-xr-x 2 root root 4096 Oct 13 14:06 srv
dr-xr-xr-x 13 root root 0 Jan 11 14:36 sys
drwxrwxrwt 2 root root 4096 Oct 13 14:12 tmp
drwxr-xr-x 1 root root 4096 Oct 13 14:06 usr
drwxr-xr-x 11 root root 4096 Oct 13 14:12 var
root@f7711ff91c90:/# tulis /bin
lrwxrwxrwx 1 root root 7 Apr 22 2024 /bin -> usr/bin
root@f7711ff91c90:/# tulis /bin | head
lrwxrwxrwx 1 root root 7 Apr 22 2024 /bin -> usr/bin
root@f7711ff91c90:/# █
```

4. Task 4 Create a Python Container

4.1 Python Program Preparation

A new directory was created on the Ubuntu virtual machine to store the files for this task. Inside this directory a simple Python program was written using a text editor. The program was executed locally using the Python interpreter to verify that it worked correctly before containerization.

Figure 9 Python script and Dockerfile for container

```
varrelomar@ubuntu1:~/task4$ pwd
/home/varrelomar/task4
varrelomar@ubuntu1:~/task4$ cat > hello.py <<'EOF'
> print("PrabowoGemoy")
> EOF
varrelomar@ubuntu1:~/task4$ python3 hello.py
PrabowoGemoy
varrelomar@ubuntu1:~/task4$ cat > DockerFile <<'EOF'
> FROM python:3.11-slim
> WORKDIR /hello
> COPY hello.py
> CMD ["python", "hello.py"]
> EOF
varrelomar@ubuntu1:~/task4$ ls -l
total 8
-rw-rw-r-- 1 varrelomar varrelomar 78 Jan 11 15:21 DockerFile
-rw-rw-r-- 1 varrelomar varrelomar 22 Jan 11 15:18 hello.py
varrelomar@ubuntu1:~/task4$ cat DockerFile
FROM python:3.11-slim
WORKDIR /hello
COPY hello.py
CMD ["python", "hello.py"]
```

4.2 Dockerfile Creation and Image Build

A Dockerfile was created to package the Python program into a Docker image. The Dockerfile used a lightweight Python base image and copied the Python script into the container working directory. The default command was set to execute the Python program when the container starts.

The Docker image was built successfully using the docker build command. The build output confirmed that all required layers were created without errors.

Figure 10 Docker build successful for Python container

```
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  3.072kB
Step 1/4 : FROM python:3.11-slim
3.11-slim: Pulling from library/python
2ae15a201602: Pulling fs layer
d6c334210834: Pulling fs layer
e520b56ca06c: Pulling fs layer
fadbb2bb3227: Pulling fs layer
fadbb2bb3227: Waiting
d6c334210834: Verifying Checksum
d6c334210834: Download complete
fadbb2bb3227: Verifying Checksum
fadbb2bb3227: Download complete
e520b56ca06c: Verifying Checksum
e520b56ca06c: Download complete
2ae15a201602: Verifying Checksum
2ae15a201602: Download complete
2ae15a201602: Pull complete
d6c334210834: Pull complete
e520b56ca06c: Pull complete
fadbb2bb3227: Pull complete
Digest: sha256:1dd3dca85e22886e44fcad1bb7ccab6691dfa83db52214cf9e20696e095f3e36
Status: Downloaded newer image for python:3.11-slim
--> e1523b812f94
Step 2/4 : WORKDIR /hello
--> Running in 290cf9a70b06
--> Removed intermediate container 290cf9a70b06
--> 250c5e0a0584
Step 3/4 : COPY hello.py .
--> 58142747fbb6
Step 4/4 : CMD ["python", "hello.py"]
--> Running in 32782ba3a97f
--> Removed intermediate container 32782ba3a97f
--> f399ac2d2631
Successfully built f399ac2d2631
Successfully tagged task4:latest
```

4.3 Running the Python Container

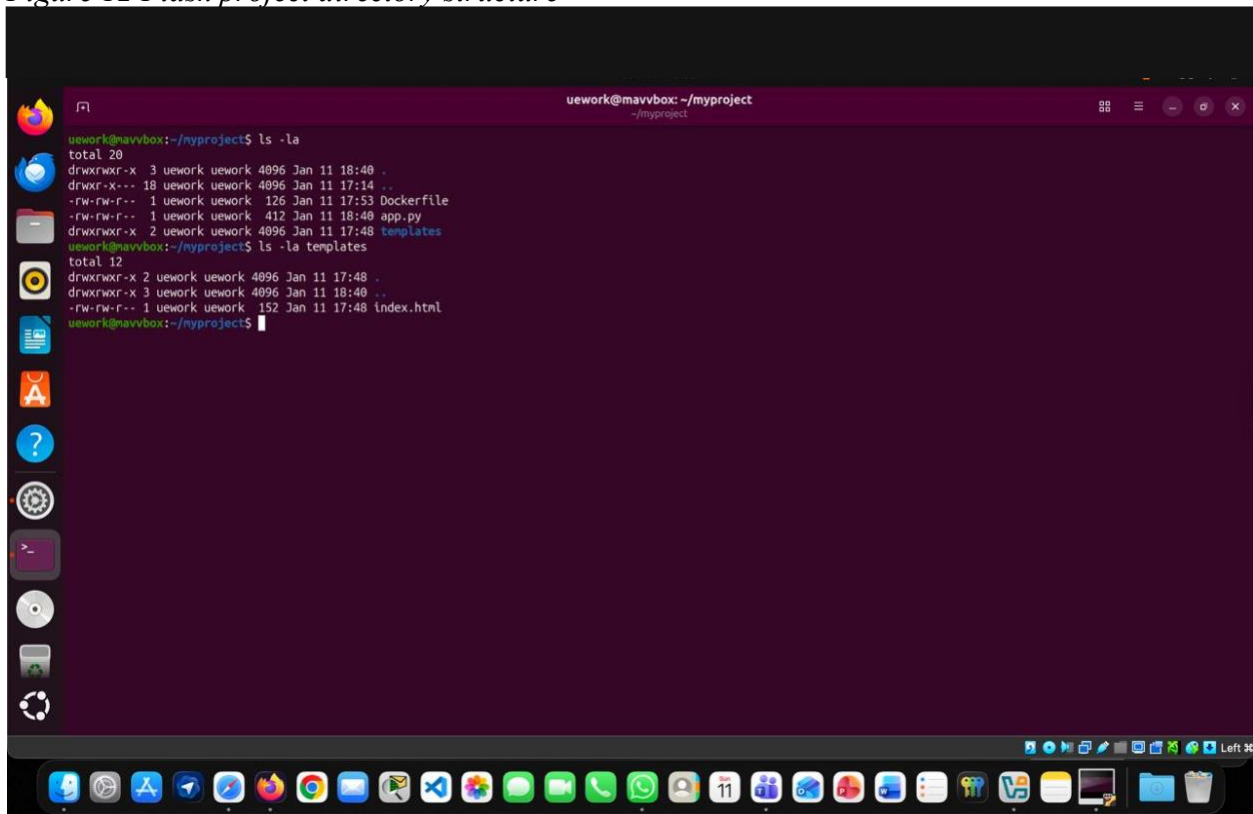
The built Docker image was executed using the `docker run` command. The container ran the Python program automatically and printed the expected output to the terminal. This confirmed that the Python application was successfully containerized and executed inside Docker.

5. Task 5 Flask Python Container with Template

5.1 Project Structure Setup

A new project directory was created for the Flask application. Inside this directory an application file and a templates directory were created. The templates directory contained an HTML file used by the Flask application.

Figure 12 Flask project directory structure

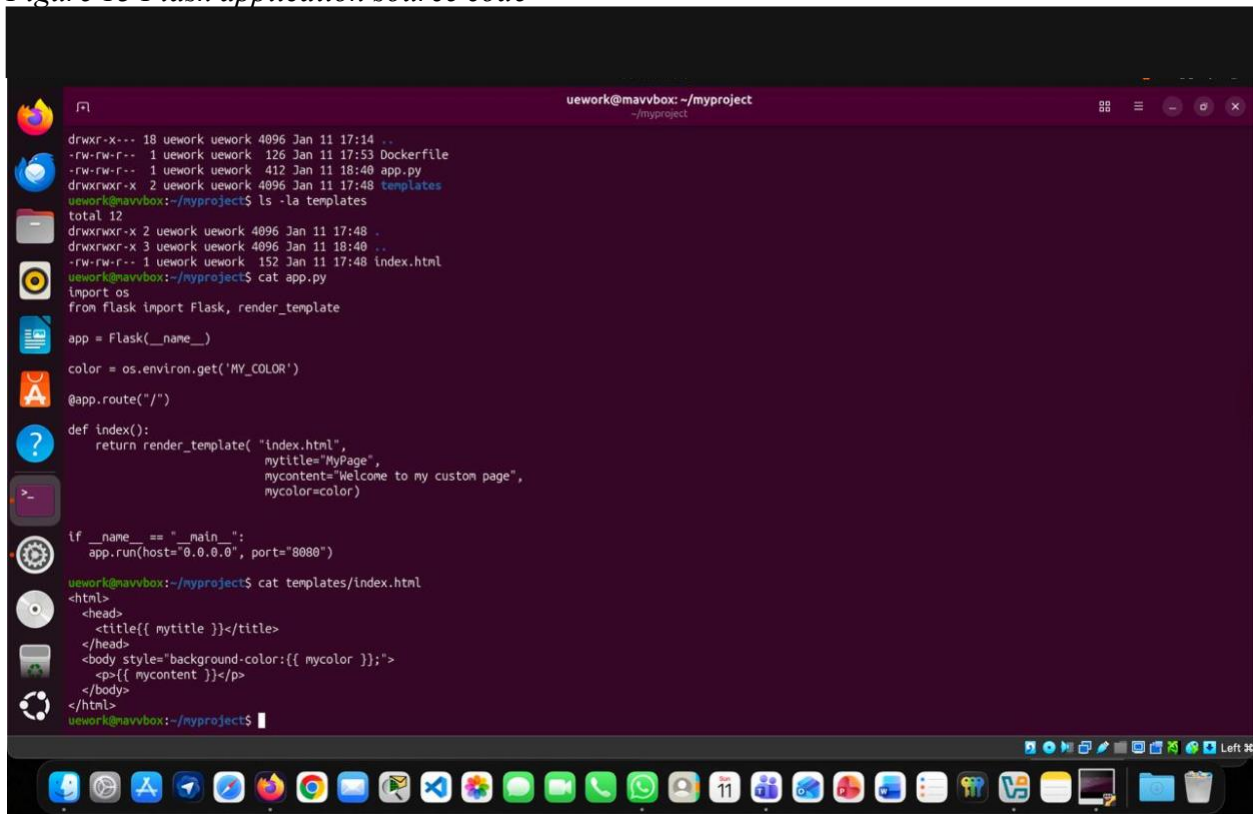
A terminal window titled 'uework@mavvbox: ~/myproject' with a dark purple background. The terminal shows the output of 'ls -la' and 'ls -la templates'. The first command shows a directory with 20 total size, containing files like 'Dockerfile', 'app.py', and 'templates'. The second command shows the 'templates' directory with 12 total size, containing 'Index.html'. The terminal is part of a desktop environment with a dock at the bottom and a sidebar on the left.

```
uework@mavvbox:~/myproject
uework@mavvbox:~/myproject$ ls -la
total 20
drwxrwxr-x 3 uework uework 4096 Jan 11 18:40 .
drwxr-x--- 18 uework uework 4096 Jan 11 17:14 ..
-rw-rw-r-- 1 uework uework 126 Jan 11 17:53 Dockerfile
-rw-rw-r-- 1 uework uework 412 Jan 11 18:40 app.py
drwxrwxr-x 2 uework uework 4096 Jan 11 17:48 templates
uework@mavvbox:~/myproject$ ls -la templates
total 12
drwxrwxr-x 2 uework uework 4096 Jan 11 17:48 .
drwxrwxr-x 3 uework uework 4096 Jan 11 18:40 ..
-rw-rw-r-- 1 uework uework 152 Jan 11 17:48 Index.html
uework@mavvbox:~/myproject$
```

5.2 Flask Application Implementation

The Flask application was implemented in Python. The application renders an HTML template and reads an environment variable named `MY_COLOR`. This variable is used to dynamically set the background color of the web page.

Figure 13 Flask application source code

A terminal window titled 'uework@mavvbox: ~/myproject' displays the source code for a Flask application. The code includes file permissions, imports for Flask and os, app initialization, a route for '/', and an index function that renders a template. Below the code, the terminal shows the output of 'ls -la templates' and 'cat templates/index.html'. The file listing shows 'app.py' and 'index.html' in the 'templates' directory. The HTML file content is an index page with a title and a paragraph. The terminal window has a dark purple background and a dock at the bottom with various application icons.

```
drwxr-x--- 18 uework uework 4096 Jan 11 17:14 ..
-rw-rw-r-- 1 uework uework 126 Jan 11 17:53 Dockerfile
-rw-rw-r-- 1 uework uework 412 Jan 11 18:40 app.py
drwxrwxr-x 2 uework uework 4096 Jan 11 17:48 templates
uework@mavvbox:~/myproject$ ls -la templates
total 12
drwxrwxr-x 2 uework uework 4096 Jan 11 17:48 .
drwxrwxr-x 3 uework uework 4096 Jan 11 18:40 ..
-rw-rw-r-- 1 uework uework 152 Jan 11 17:48 index.html
uework@mavvbox:~/myproject$ cat app.py
import os
from flask import Flask, render_template

app = Flask(__name__)

color = os.environ.get('MY_COLOR')

@app.route("/")

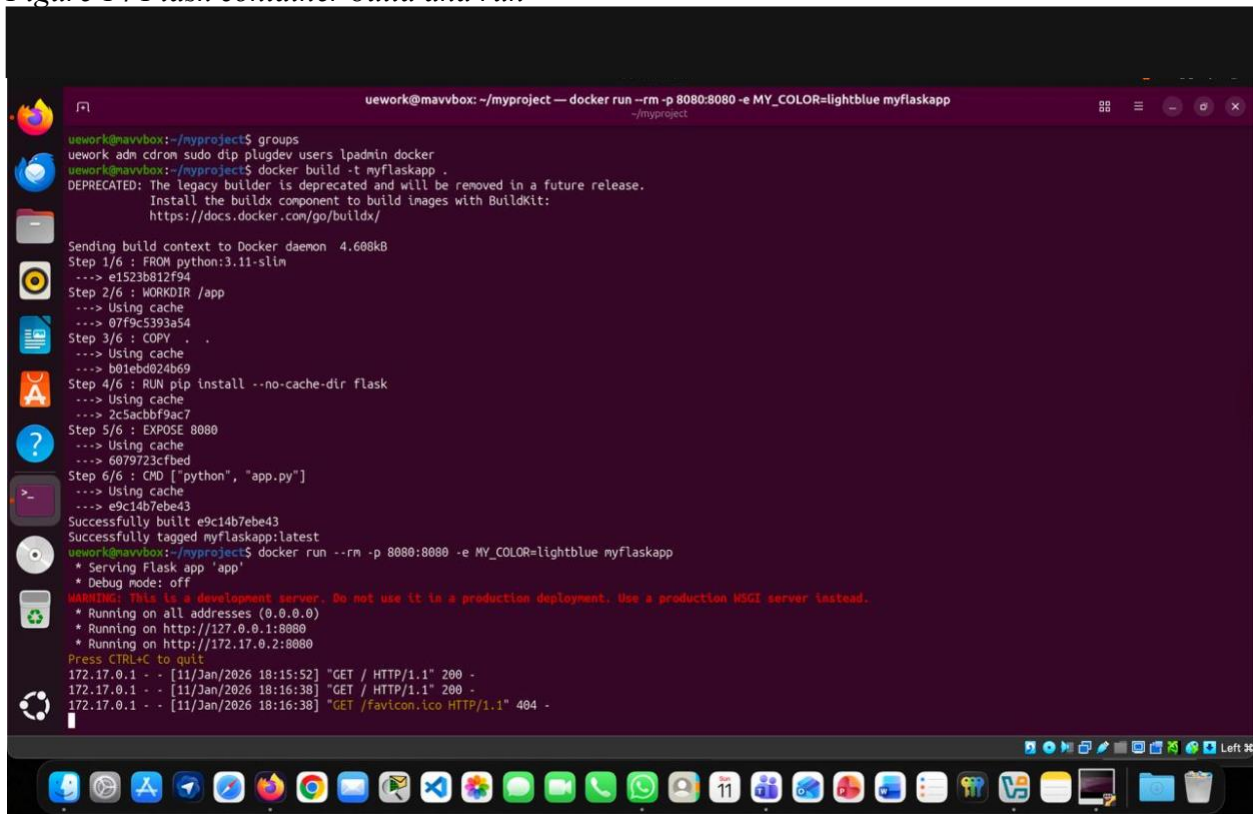
def index():
    return render_template("index.html",
                           mytitle="MyPage",
                           mycontent="Welcome to my custom page",
                           mycolor=color)

if __name__ == "__main__":
    app.run(host="0.0.0.0", port="8080")
uework@mavvbox:~/myproject$ cat templates/index.html
<html>
<head>
<title{{ mytitle }}</title>
</head>
<body style="background-color:{{ mycolor }};">
<p>{{ mycontent }}</p>
</body>
</html>
uework@mavvbox:~/myproject$
```

5.3 Dockerfile Creation Build and Run

A Dockerfile was created to containerize the Flask application. The Dockerfile installed the required Python dependencies and copied the application files into the container. The container was configured to expose port 8080.

Figure 14 Flask container build and run

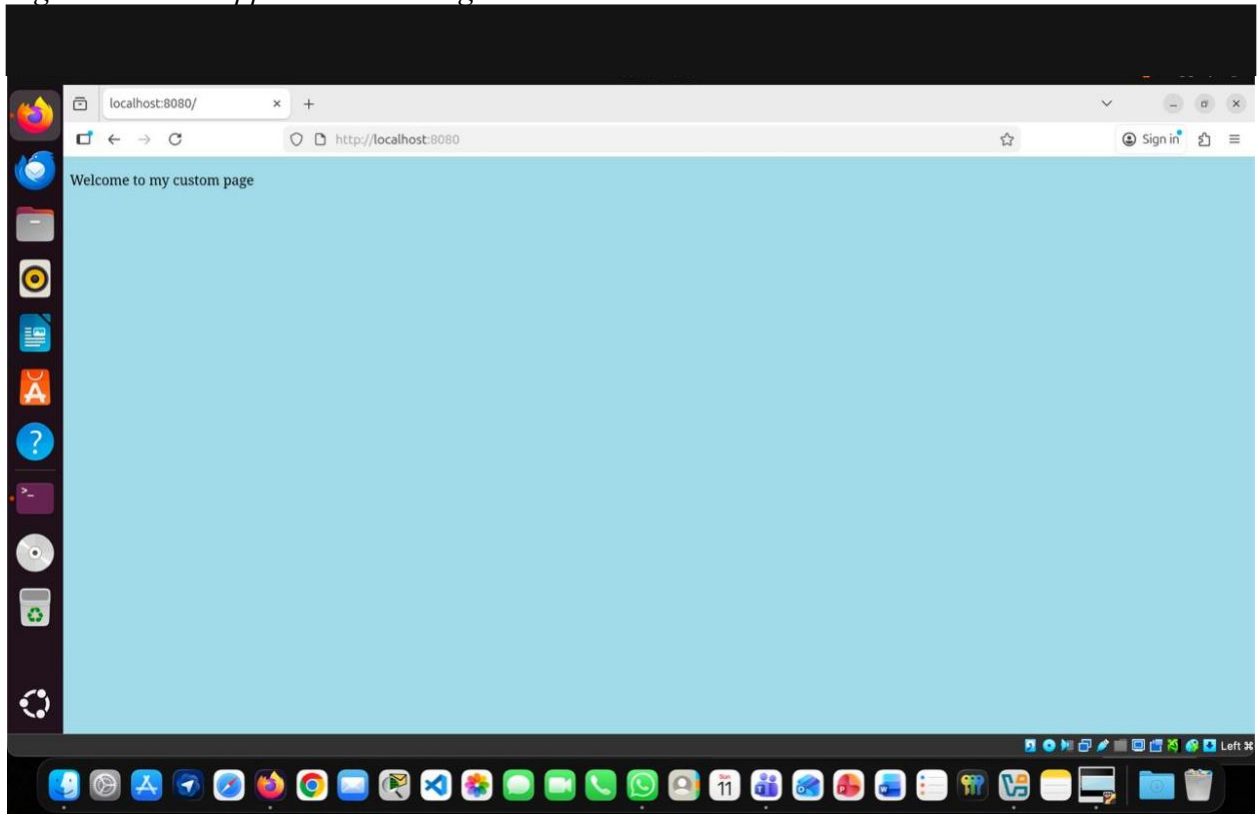
A terminal window with a dark purple background and light blue text. The window title is 'uework@mavvbox: ~/myproject — docker run --rm -p 8080:8080 -e MY_COLOR=lightblue myflaskapp'. The terminal shows the following commands and output:
uework@mavvbox:~/myproject\$ groups
uework adm cdrom sudo dip plugdev users lpadmin docker
uework@mavvbox:~/myproject\$ docker build -t myflaskapp .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/
Sending build context to Docker daemon 4.688kB
Step 1/6 : FROM python:3.11-slim
--> e1523b812f94
Step 2/6 : WORKDIR /app
--> Using cache
--> 07f9c5393a54
Step 3/6 : COPY . .
--> Using cache
--> b01ebd024b69
Step 4/6 : RUN pip install --no-cache-dir flask
--> Using cache
--> 2c5acbbf9ac7
Step 5/6 : EXPOSE 8080
--> Using cache
--> 6079723cfbd
Step 6/6 : CMD ["python", "app.py"]
--> Using cache
--> e9c14b7ebe43
Successfully built e9c14b7ebe43
Successfully tagged myflaskapp:latest
uework@mavvbox:~/myproject\$ docker run --rm -p 8080:8080 -e MY_COLOR=lightblue myflaskapp
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8080
* Running on http://172.17.0.2:8080
Press CTRL+C to quit
172.17.0.1 - - [11/Jan/2026 18:15:52] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [11/Jan/2026 18:16:38] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [11/Jan/2026 18:16:38] "GET /Favicon.ico HTTP/1.1" 404 -
The terminal window has a dock at the bottom with various application icons.

The Docker image was built successfully. The container was then executed with port mapping enabled and the MY_COLOR environment variable set.

5.4 Browser Verification

The running Flask application was accessed through a web browser using the local host address and port 8080. The web page loaded successfully and displayed the expected content with the background color defined by the environment variable.

Figure 15 Flask application running in browser



6. Conclusion

All tasks from Task 2 to Task 5 were completed successfully. The project demonstrated the installation and use of Linux virtual machines user and package management Docker installation and usage Python application containerization and deployment of a Flask based web application. Each task was verified through command line outputs container execution results and browser-based validation.